

What Are Functional Interfaces?

A **functional interface** is an interface in Java that has only **one abstract method**. It can contain other methods (like default or static methods), but only one abstract method. These interfaces are the backbone of functional programming in Java and enable the use of **lambda expressions** to write concise, functional-style code.

The key built-in functional interfaces in `java.util.function` are:

1. `Function<T, R>`: Takes a single input of type T and returns a result of type R.
2. `Predicate<T>`: Evaluates a boolean condition on an input of type T.
3. `Consumer<T>`: Performs an operation on an input of type T without returning a result.
4. `Supplier<T>`: Supplies a result of type T without taking any input.
5. `BiFunction<T, U, R>`: Takes two inputs of types T and U and returns a result of type R.
6. `BiConsumer<T, U>`: Performs an operation on two inputs, T and U, without returning a result.

Why Use Functional Interfaces?

1. Simplify code by reducing boilerplate with **lambda expressions**.
2. Enable declarative-style programming with **Streams API**.
3. Enhance readability and modularity.
4. Leverage existing functional programming patterns for common tasks like filtering, transforming, and consuming data.

POC Outline: Real-World Scenario

Scenario:

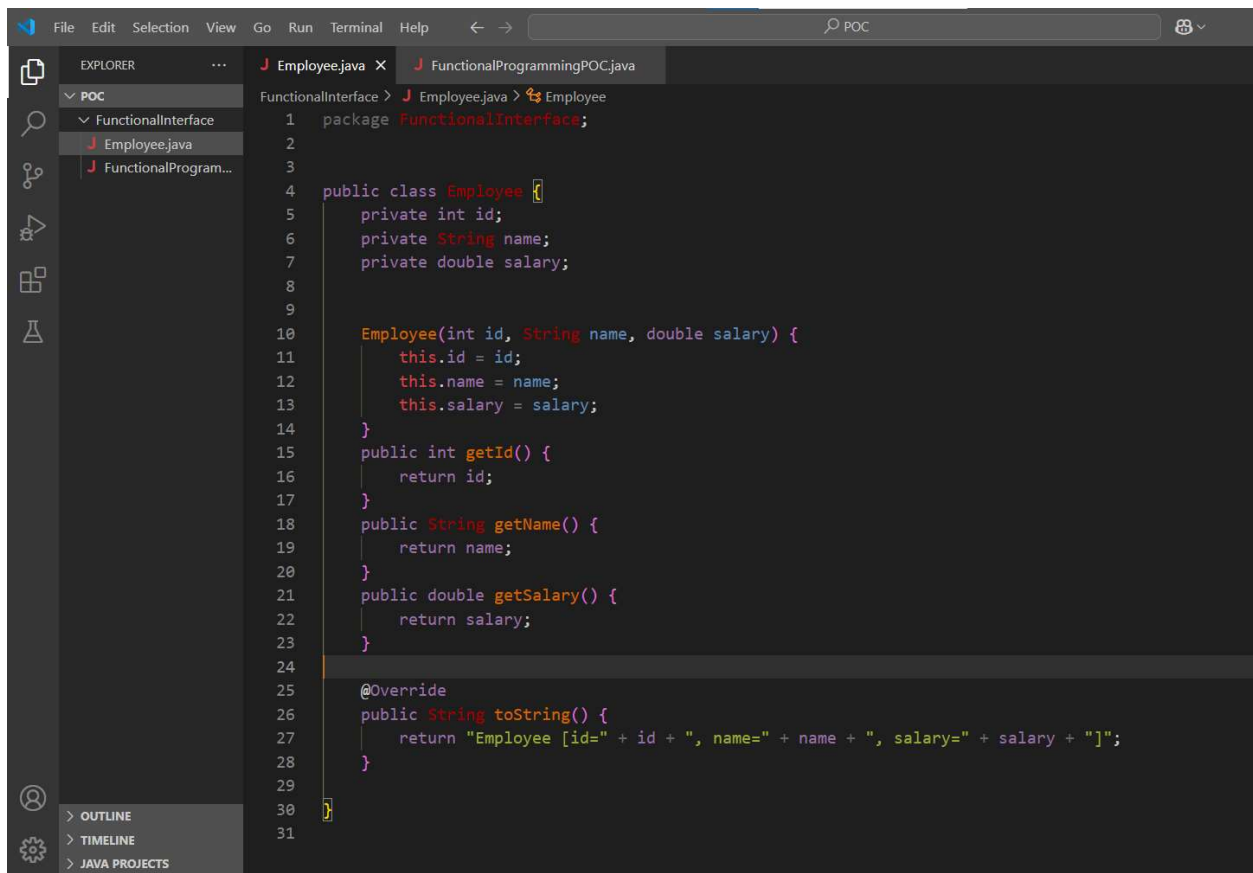
You are tasked with processing employee data. Operations include:

1. Filtering employees based on conditions (e.g., salary).
2. Converting salaries to another currency.
3. Generating placeholder or default employee data.
4. Performing actions like printing details or updating information.

POC Walkthrough

Step 1: Employee Class

We'll start by modeling the Employee class to represent the objects we will manipulate.



```
1 package FunctionalInterface;
2
3
4 public class Employee {
5     private int id;
6     private String name;
7     private double salary;
8
9
10    Employee(int id, String name, double salary) {
11        this.id = id;
12        this.name = name;
13        this.salary = salary;
14    }
15    public int getId() {
16        return id;
17    }
18    public String getName() {
19        return name;
20    }
21    public double getSalary() {
22        return salary;
23    }
24
25    @Override
26    public String toString() {
27        return "Employee [id=" + id + ", name=" + name + ", salary=" + salary + "];"
28    }
29
30
31 }
```

2. Employee Object Creation:

A list of Employee objects is created using ArrayList. Each employee has:

- id: A unique identifier.
- name: Represents the employee's name.
- salary: The employee's salary.

```
1 package FunctionalInterface;
2
3 import java.util.ArrayList;
4 import java.util.List;
5 import java.util.function.Consumer;
6 import java.util.function.Function;
7 import java.util.function.Predicate;
8 import java.util.function.Supplier;
9
10 public class FunctionalProgrammingPOC {
11     public static void main(String[] args) {
12         List<Employee> employees = new ArrayList<>();
13         employees.add(new Employee(id:1, name:"Shankar", salary:5000));
14         employees.add(new Employee(id:2, name:"Sumith", salary:60000));
15         employees.add(new Employee(id:3, name:"Shiva", salary:70000));
16         employees.add(new Employee(id:4, name:"Sourabh", salary:80000));
17
18         //Predicate to filter employees with salary greater than 60000
19         Predicate<Employee> higherSalaryPredicate = emp -> emp.getSalary() > 60000;
20         System.out.println(x:"Employees with salary grater than 60000:");
21         employees.stream().filter(higherSalaryPredicate).forEach(System.out::println);
22
23         // 2. Function: Convert salary to another currency (e.g., USD to INR conversion)
24         // Assuming 1 USD = 75 INR for this example
25
26         Function<Employee, Double> convertSalary = emp -> emp.getSalary() * 82.5;
27         System.out.println(x:"\nSalaries converted to INR:");
28         employees.stream().map(emp -> convertSalary.apply(emp)).forEach(System.out::println);
29
30         // 3. Supplier: Generate a default employee
31         Supplier<Employee> defaultEmployeeSupplier = () -> new Employee(id:0, name:"Default Employee", salary:0);
32         System.out.println("\nDefault Employee: " + defaultEmployeeSupplier.get());
33
34         // 4. Consumer: Print employee details
35         Consumer<Employee> printEmployee = emp -> System.out.println("Employee: " + emp);
36         System.out.println(x:"\nEmployee Details:");
37         employees.forEach(printEmployee);
38
39     }
40
41 }
42
```

Four employees are added with the following details:

1. **Employee 1:** id=1, name="Shankar", salary=5000
2. **Employee 2:** id=2, name="Sumith", salary=6000
3. **Employee 3:** id=3, name="Shiva", salary=7000
4. **Employee 4:** id=4, name="Sourabh", salary=8000

3. Purpose of the Code:

The code is structured to show how functional programming can simplify operations, such as:

- Filtering employees based on conditions using Predicate.
- Transforming data (e.g., salary) using Function.

- Providing default values using `Supplier`.
- Consuming data like printing employee details using `Consumer`.

Applications:

This example can be extended with tasks like:

- Filtering employees whose salary exceeds a specific threshold.
- Calculating bonuses using functions.
- Generating default employee profiles.

Step 2: Demonstrate Predicate (Filtering Data)

A **Predicate** evaluates a boolean condition on an object. For example, filter employees whose salaries are greater than 60,000.

```
import java.util.function.Predicate;

Predicate<Employee> highSalaryPredicate = emp -> emp.getSalary() > 60000;

System.out.println("Employees with salary greater than 60,000:");
employees.stream()
    .filter(highSalaryPredicate) // Apply the Predicate
    .forEach(System.out::println); // Print the filtered employees
```

Output:

```
Employees with salary greater than 60000:
Employee [id=3, name=Shiva, salary=70000.0]
Employee [id=4, name=Sourabh, salary=80000.0]
```

Step 3: Demonstrate Function (Transforming Data)

A **Function** takes an input, performs an operation, and returns a result. For example, convert an employee's salary from USD to INR.

```
import java.util.function.Function;

Function<Employee, Double> convertSalary = emp -> emp.getSalary() * 82.5;

System.out.println("\nSalaries in INR:");
employees.stream()
    .map(convertSalary) // Apply the Function
    .forEach(System.out::println); // Print the converted salaries
```

OUTPUT:

```
Salaries converted to INR:
412500.0
4950000.0
5775000.0
6600000.0
```

Step 4: Demonstrate Supplier (Generating Data)

A **Supplier** provides a result without taking any input. For example, generate a default Employee.

```
import java.util.function.Function;

Function<Employee, Double> convertSalary = emp -> emp.getSalary() * 82.5;

System.out.println("\nSalaries in INR:");
employees.stream()
    .map(convertSalary) // Apply the Function
    .forEach(System.out::println); // Print the converted salaries
```

OUTPUT:

```
Default Employee: Employee [id=0, name=Default Employee, salary=0.0]
```

Step 5: Demonstrate Consumer (Performing Actions)

A **Consumer** performs an action on an object without returning a result. For example, print employee details.

```
Default Employee: Employee [id=0, name=Default Employee, salary=0.0]

Employee Details:
Employee: Employee [id=1, name=Shankar, salary=5000.0]
Employee: Employee [id=2, name=Sumith, salary=60000.0]
Employee: Employee [id=3, name=Shiva, salary=70000.0]
Employee: Employee [id=4, name=Sourabh, salary=80000.0]
PS C:\Users\SHANKAR\Desktop\POC>
```

Conclusion

By combining functional interfaces like Predicate, Function, Supplier, and Consumer, your POC can demonstrate:

- Filtering and processing data with Predicate.
- Transforming data using Function.
- Generating values with Supplier.
- Performing side effects using Consumer.

This POC will showcase the real-world power of **functional programming** in Java 8. Let me know if you'd like additional examples or more explanations!